

Shor's Algorithm as Quantum Circuits

*How factoring and the elliptic-curve discrete logarithm are built, run and costed
— from the textbook circuit to the optimised ones, in Qiskit*

How to read these slides

Colours carry meaning

Blue — the counting register, the one we measure.

Vermillion — the target register, where the arithmetic happens.

Purple — what we are after: the period r , the key k , the angles they produce.

Colour is a reminder, never the only carrier of information.

Each act is one question and its answer

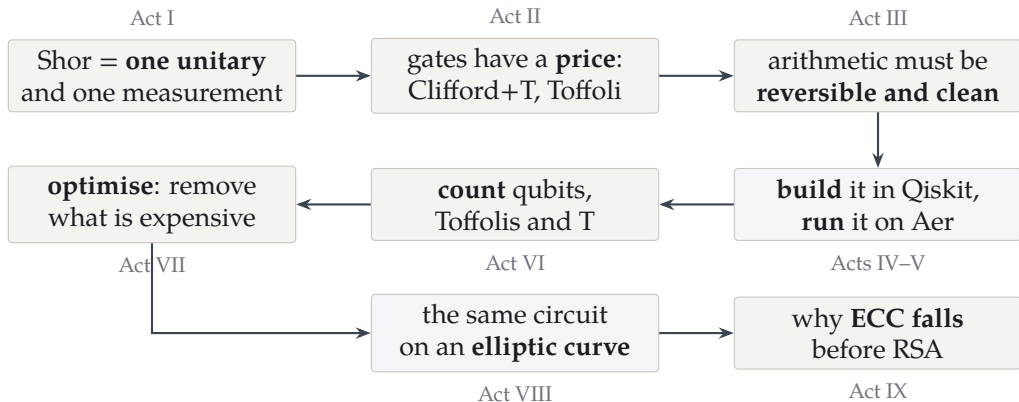
Difficulty. Opens each act: the obstacle to get past.

Resolved. Closes the act: how it was removed.

Costs
qubits, Toffolis, T gates

Every number is computed by the code in `shor_qiskit/`, as in `notebooks/shor_walkthrough.ipynb`.

The road map, in plain words



The quantum part of Shor is short. Almost all of its cost is reversible arithmetic — that is what we build, run and count.

Act I — Shor's algorithm is one unitary and one measurement

Act I — factoring reduces to finding a period

Difficulty 1. To factor N , pick a with $\gcd(a, N) = 1$ and find its **order** r : the smallest $r > 0$ with $a^r \equiv 1 \pmod{N}$.

x	0	1	2	3	4	5	...
$7^x \bmod 15$	1	7	4	13	1	7	...

 $\Rightarrow r = 4$

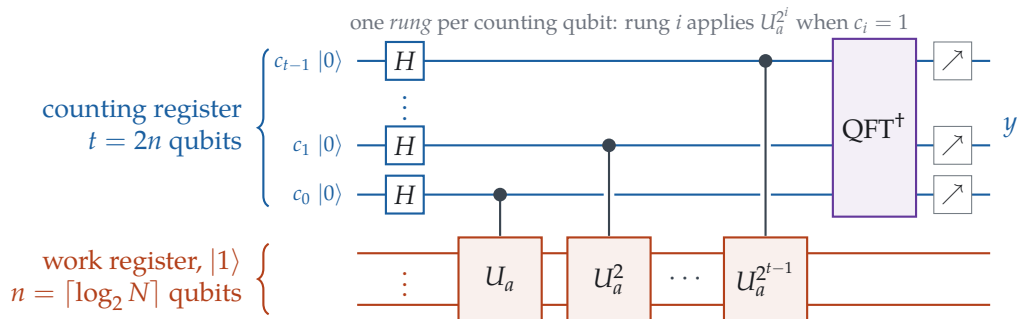
Why the period factors N : if r is even, N divides

$$a^r - 1 = (a^{r/2} - 1)(a^{r/2} + 1),$$

and unless $a^{r/2} \equiv -1$ each factor shares a divisor with N : $\gcd(48, 15) = 3$,
 $\gcd(50, 15) = 5$.

The period is a property of the whole sequence, not of any one value. Only the period needs a quantum computer; the rest is Euclid.

Phase estimation: the circuit



$$U_{\text{Shor}} = (\text{QFT}^\dagger \otimes I) C-U_a^{2^{t-1}} \dots C-U_a^2 C-U_a (H^{\otimes t} \otimes I) \text{ applied to } |0\rangle^{\otimes t} |1\rangle$$

where $C-U = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes U$, and $U_a^{2^i} = U_{a^{2^i \bmod N}}$ is computed classically.

Why the measurement reveals s/r

1. **Eigenvectors.** $U_a|u_s\rangle = e^{2\pi i \varphi}|u_s\rangle$, $\varphi = s/r$; the input is $|1\rangle = \frac{1}{\sqrt{r}} \sum_s |u_s\rangle$.
2. **Kickback.** Rung j leaves $|u_s\rangle$ alone and turns its control by $2^j \varphi$. The counting register ($Q = 2^t$) becomes a product:

$$\bigotimes_j \frac{|0\rangle + e^{2\pi i 2^j \varphi} |1\rangle}{\sqrt{2}} = \frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} e^{2\pi i \varphi x} |x\rangle =: |\Phi_\varphi\rangle$$

since branch x collects the turns of its 1-bits, $\prod_j e^{2\pi i 2^j \varphi x_j} = e^{2\pi i \varphi x}$: a *phase ramp* of slope φ .

3. **Readout.** With $\langle y | \text{QFT}^\dagger | x \rangle = e^{-2\pi i xy/Q} / \sqrt{Q}$:

$$\langle y | \text{QFT}^\dagger | \Phi_\varphi \rangle = \frac{1}{Q} \sum_x e^{-2\pi i xy/Q} e^{2\pi i \varphi x} = \frac{1}{Q} \sum_{x=0}^{Q-1} e^{2\pi i (\varphi - y/Q) x} = \begin{cases} \approx 1, & y \approx Q\varphi \\ \approx 0, & \text{otherwise} \end{cases}$$

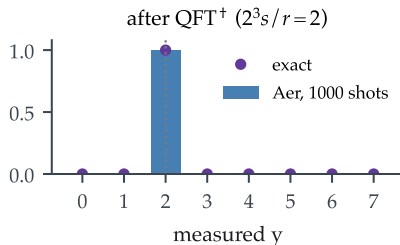
Each $|u_s\rangle$ carries its own ramp; they do not interfere: $y \approx Qs/r$, s uniformly random.

QFT[†] in action: a ramp becomes an integer

$t = 3$ counting qubits, $s/r = 1/4$: the phase $e^{2\pi i x/4} = i^x$ turns a quarter turn per step,

$$\frac{1}{\sqrt{8}} \sum_{x=0}^7 e^{2\pi i x/4} |x\rangle = \frac{|0\rangle + i|1\rangle - |2\rangle - i|3\rangle + |4\rangle + \dots}{\sqrt{8}} = \text{QFT } |2\rangle \xrightarrow{\text{QFT}^\dagger} |2\rangle$$

the ramp: amplitude of $|x\rangle$ turns 90° per step ($s/r = 1/4$)



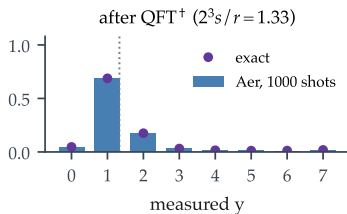
Every shot gives $y = 2$, so $s/r = y/2^t = 2/8 = 1/4$: the ramp's slope, read as an integer.

When Qs/r is not an integer: a peak at the nearest one

$$\langle y | \text{QFT}^\dagger | \Phi_{s/r} \rangle = \frac{1}{Q} \sum_{x=0}^{Q-1} e^{2\pi i (s/r - y/Q) x}$$

A sum of Q unit arrows: they all align (sum = 1) only when $y = Qs/r$; otherwise they cancel, least for the integers nearest to it.

the ramp: amplitude of $|x\rangle$ turns 60° per step ($s/r = 1/6$)



$s/r = 1/6$: $y = 1$ with probability 0.688 (measured 0.691) — above the guaranteed $4/\pi^2$.

The readout in Qiskit

```
import numpy as np
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from shor_essentials import qft

t, phi = 3, 1 / 4                                # s/r = 1/4, t = 3
qc = QuantumCircuit(t, t)
qc.h(range(t))
for q in range(t):                                # the ramp the rungs leave:
    qc.p(2 * np.pi * phi * 2**q, q) # qubit q turns 2^q s/r
qc.append(qft(t).inverse(), range(t)) # ramp -> integer
qc.measure(range(t), range(t))
sim = AerSimulator()
print(sim.run(transpile(qc, sim), shots=1000).result().get_counts())
```

prints {'010': 1000}:
every shot is $y = 2$.

The phase gates build the
ramp that the controlled
rungs would leave;
`qft(t).inverse()` is the same
readout as in the full circuit.

The same pattern breaks elliptic-curve keys

RSA: factoring

group: $(\mathbb{Z}_{N'}^*, \times)$

box: $U_a|x\rangle = |ax \bmod N\rangle$

unknown: the order r

one counting register

ECDSA / ECDH: elliptic-curve discrete log

group: points of a curve, with $+$

box: $A_R|X\rangle = |X + R\rangle$

unknown: the key k in $Q = kP$

two counting registers

Both are **phase estimation of a group operation**. Only the box changes: a modular multiplication, or a point addition (Act VIII).

***Resolved.** Shor is one fixed circuit pattern. Everything that follows is about building, running and paying for the box.*

Act II — which gates, and what they cost

Act II — a fault-tolerant computer has a small gate set

Difficulty 2. What does each gate cost on an error-corrected computer?

Clifford gates: cheap

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}, \quad \text{CNOT}$$

Cheap on the **surface code**, the standard error-correcting code; alone, classically simulable (Gottesman–Knill).

The T gate: expensive

$$T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$$

Makes the set **universal**. No code has a universal set of transversal gates (Eastin–Knill): on the surface code each T consumes a **magic state** $T|+\rangle$, distilled 15-to-1 in factories.

The cost model: count T gates. Cliffords are almost free by comparison.

The Toffoli: a reversible AND

$$\text{Toffoli } |a, b, c\rangle = |a, b, c \oplus ab\rangle$$

Reversible: it permutes basis states

It swaps $|110\rangle \leftrightarrow |111\rangle$ and fixes the other six. A permutation matrix is unitary; its inverse is the reverse permutation, here the Toffoli itself:

$$c \oplus ab \oplus ab = c.$$

What it costs

In Clifford+T, a Toffoli is **7 T gates**. A classical function is therefore priced exactly by its Toffoli count.

Reversible arithmetic = Toffolis + CNOTs. Its T cost is exact: $7 \times$ the number of Toffolis.

Any function becomes reversible: keep the input

$$f: x \mapsto f(x) \quad \rightsquigarrow \quad U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$$

Why it is reversible

Applied twice, $y \oplus f(x) \oplus f(x) = y$: U_f is its own inverse, even when f has no inverse. AND sends 00, 01, 10 to 0, yet U_{AND} is the Toffoli.

A whole circuit, gate by gate

NOT and XOR are reversible already (X, CNOT). Each AND becomes a Toffoli onto a fresh $|0\rangle$ (Toffoli, 1980):

$$|x\rangle |0\rangle |0\rangle \mapsto |x\rangle |f(x)\rangle |g(x)\rangle.$$

Any classical circuit runs reversibly, at one Toffoli and one fresh bit per AND. The fresh bits keep the intermediate values $g(x)$: garbage (Act III).

Phase shifts are exact only for multiples of $\pi/4$

$$P(\theta) = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}, \quad \text{CP}(\theta) = \text{diag}(1, 1, 1, e^{i\theta}) = 2 \text{ CNOT} + 3 P(\pm\theta/2).$$

Exact

$$P(\pi) = Z, P(\pi/2) = S, P(\pi/4) = T.$$

Not exact: a “small rotation”

$P(\pi/8), P(\pi/16), \dots$: their entries are not in the ring $\mathbb{Z}[\frac{1}{\sqrt{2}}, i]$ that Clifford+T circuits generate. They must be **approximated**.

The QFT and the Fourier-space adders of Act IV are built from $\text{CP}(\pi/2^d)$ and $P(2\pi X/2^m)$ — mostly small rotations.

Solovay–Kitaev: approximation is cheap, but not free

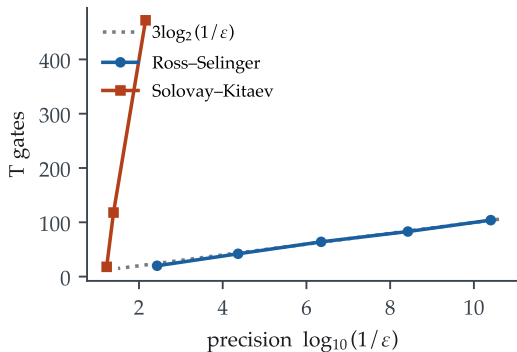
Solovay–Kitaev theorem

Any finite universal gate set closed under inverses approximates any one-qubit gate to error ε with $O(\log^c(1/\varepsilon))$ gates. Dawson–Nielsen’s construction: $c = \log 5 / \log \frac{3}{2} \approx 3.97$; later improved.

Ross–Selinger (gridsynth)

For z -rotations: $\approx 3 \log_2(1/\varepsilon)$ T gates, near-optimal.

On $P(\pi/16)$, at similar error: **20 T** (gridsynth, $\varepsilon \approx 4 \cdot 10^{-3}$) against **472 T** (Solovay–Kitaev, $7 \cdot 10^{-3}$).



The price list

gate	T gates	note
H, S, CNOT (Clifford)	0	cheap on the code
$T = P(\pi/4)$	1	one magic state
Toffoli	7	(4 as a temporary AND, Act VII)
small rotation $P(\theta)$	$\approx 3 \log_2(1/\varepsilon) \approx 60\text{--}100$	budget ε/R for each of R

Resolved. *One small rotation costs as much as about ten Toffolis. Circuits built from Toffolis have an exact, low T count; circuits built from rotations pay for synthesis on every one.*

Act III — reversible arithmetic, and why it must clean up

Act III — arithmetic leaves scratch values behind

Difficulty 3. A quantum circuit cannot erase. Computing $f(x)$ reversibly leaves intermediate values — carries, flags, partial products — in **ancillas**, scratch qubits that start in $|0\rangle$. Does this *garbage* matter if we never measure it?

A reversible circuit for f :

$$|x\rangle |0\rangle |0\rangle \longmapsto |x\rangle |f(x)\rangle |g(x)\rangle,$$

where $g(x)$ is everything else it wrote down. In Shor, x is the **counting register**, in superposition:

$$|\Psi\rangle = \frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle |f(x)\rangle |g(x)\rangle, \quad f(x) = a^x \bmod N, \quad Q = 2^t.$$

Garbage destroys the interference: the formula

Apply $\text{QFT}^\dagger$ to the **counting register** and measure y . The unmeasured registers enter through their **overlaps**:

$$P(y) = \frac{1}{Q^2} \sum_{x,x'} e^{2\pi i (x'-x)y/Q} \underbrace{\langle f(x') | f(x) \rangle}_{\text{target}} \underbrace{\langle g(x') | g(x) \rangle}_{\text{garbage}}.$$

No garbage — or garbage that depends only on $f(x)$

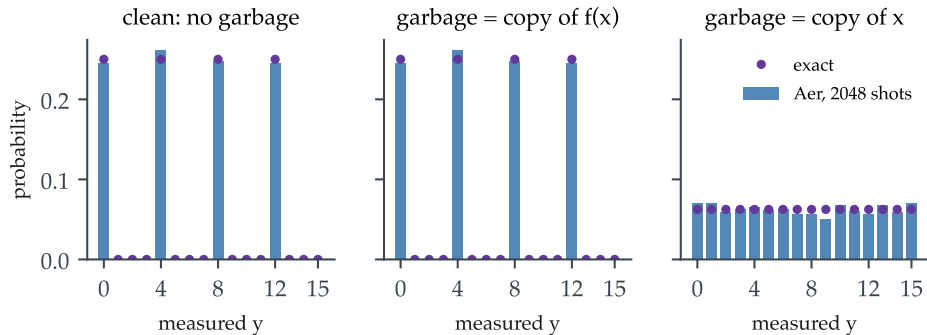
The overlaps are $[x \equiv x' \bmod r]$: each period's terms interfere, and $P(y)$ peaks at $y \approx sQ/r$.

Garbage that records x

$\langle g(x') | g(x) \rangle = \delta_{xx'}$: every cross term vanishes, and $P(y) = \frac{1}{Q^2} \cdot Q = \frac{1}{Q}$ — flat.

Garbage is a which-path record. If it tells x apart beyond $f(x)$, it measures x — and the interference is gone.

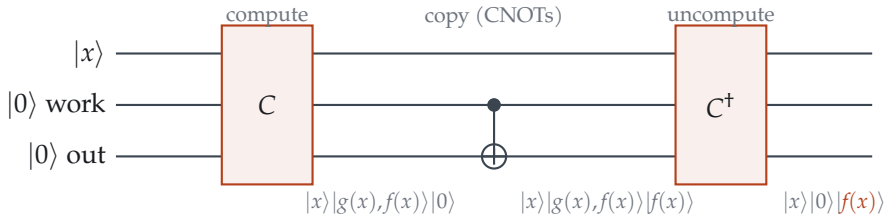
The formula on Aer: $N = 15, a = 7, t = 4$ counting qubits



Aer, Qiskit's simulator: exact probabilities (dots), sampled shots (bars). Each panel adds one line after the multiplications.

A copy of $f(x)$ changes nothing. A copy of x gives exactly $P(y) = 1/16$: the peaks are gone.

Bennett's fix: compute, copy, uncompute



The price

About twice the gates, plus a register for the copy. That factor 2 is in every count that follows.

Where it returns

the modular adder's flag · multiply, swap, *un*-multiply · the second division of a point addition · the temporary AND

Resolved. Every scratch register must be back in $|0\rangle$ before the QFT. Bennett's pattern does it, at the price of running the computation backwards.

Act IV — building the factoring circuit, level by level

Act IV — Qiskit in a few idioms

```
from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
from qiskit import transpile
from qiskit_aer import AerSimulator

def add_one(qc, y):
    # a builder appends gates
    for i in reversed(range(1, len(y))):
        qc.mcx(y[:i], y[i])      # carry: multi-controlled X
        qc.x(y[0])

y, out = QuantumRegister(3, "y"), ClassicalRegister(3, "out")
qc = QuantumCircuit(y, out)
qc.x([y[0], y[1]])              # y = 3, qubit 0 = lowest bit
add_one(qc, y)                  # y = 4
inc = QuantumCircuit(3)
add_one(inc, inc.qubits)
qc.append(inc.to_gate().inverse(), y) # run backwards: y = 3
qc.measure(y, out)
sim = AerSimulator()
print(sim.run(transpile(qc, sim), shots=100).result().get_counts())
```

prints {'011': 100}

Idioms used throughout

builders: functions that
append gates to a circuit

mcx, mcp: multi-controlled X
and phase

.inverse(): run backwards —
uncomputation

to_gate(): a sub-circuit as one
box

counts: bit strings, **qubit 0**
rightmost

Five levels, each a product of the one below

Difficulty 4. How does “multiply by a modulo N ” become gates? Build it in levels, and check each level exactly before using it.

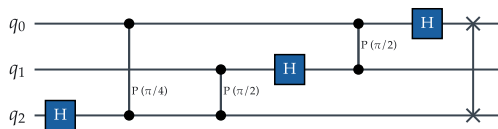
L5	U_{Shor} : t controlled rungs, then $\text{QFT}^\dagger$	order_circuit
L4	C- U_a : multiply, controlled swap, un-multiply	c_ua
L3	CMULT(a): n controlled modular additions of $a2^i$	c_mult_acc
L2	ADD $_N(X)$: seven blocks of plain additions	c_add_mod
L1	Φ ADD(X): QFT, then phases, then $\text{QFT}^\dagger$	add_const
L0	QFT: Hadamards and controlled phases	qft

Each level is verified exhaustively — every input, every ancilla back to $|0\rangle$ — before the next is built on it.

L0 — the QFT is Hadamards and controlled phases

$$\text{QFT}|j\rangle = \bigotimes_{q=0}^{n-1} \frac{|0\rangle + e^{2\pi i j 2^q / 2^n} |1\rangle}{\sqrt{2}} \implies \text{QFT} = \text{SWAP}_{\text{rev}} B_0 \cdots B_{n-1}$$

$$B_j = \left(\prod_{k < j} \text{CP}\left(\frac{\pi}{2^{j-k}}\right) \right) H_j$$



The output is a product state: one Hadamard per qubit, and one controlled phase per lower bit.

n Hadamards, $n(n-1)/2$ controlled phases, $\lfloor n/2 \rfloor$ swaps. The swaps only undo a bit reversal (3 CNOT, 0 T).

L0 in Qiskit

```
def qft(n, approx=0):
    qc = QuantumCircuit(n, name="qft")
    for j in reversed(range(n)):
        qc.h(j)
        for k in range(j):
            if approx and (j - k) > approx:
                continue # angle too small to matter
            qc.cp(np.pi / 2 ** (j - k), k, j)
    for i in range(n // 2): # reverse the qubit order
        qc.swap(i, n - 1 - i)
    return qc.to_gate()
```

Checked

equal to the DFT matrix,
and to Qiskit's QFTGate,
for $n = 1, \dots, 6$.

Its cost

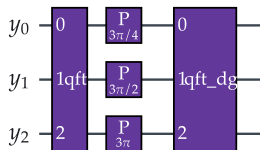
$\text{CP}(\pi/2)$ is exact (3 T).
The other controlled
phases hold small
rotations: $3\binom{n-1}{2}$ per QFT.

approx drops the smallest rotations: the approximate QFT of Act VII.

L1 — Draper's adder: addition is a phase

In the Fourier basis, qubit i of $\text{QFT}|y\rangle$ carries $e^{2\pi i y 2^i / 2^m}$. Adding X multiplies it by $e^{2\pi i X 2^i / 2^m}$: one phase per qubit, **no carries**.

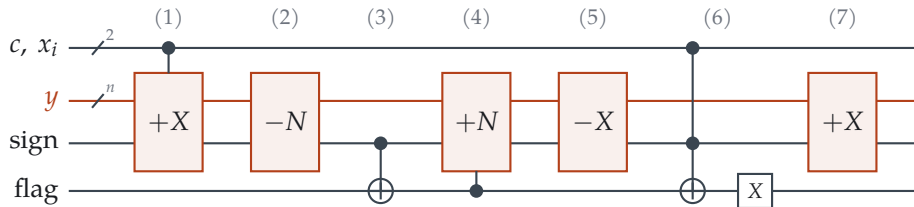
$$\Phi_{\text{ADD}}(X) = \text{QFT}^\dagger \left(\bigotimes_{i=0}^{m-1} P(2\pi X 2^i / 2^m) \right) \text{QFT}$$



$X = 3$ on 3 qubits: checked equal to the cyclic shift $y \mapsto y + 3 \bmod 8$.

To control the adder, control only the phases: the QFT pair cancels when they do not fire.

L2 — the modular adder in seven blocks



(1)–(4): add, then reduce

Add X ; subtract N ; copy the sign — did it go negative? — into the flag; add N back if so.

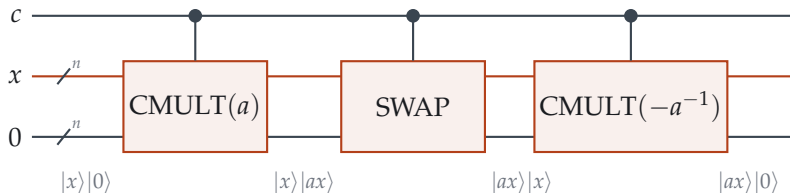
(5)–(7): uncompute the flag

Subtract X , so the sign reveals the flag; clear it; add X again. Bennett, inside one adder.

Only blocks (1) and (6) carry the outer controls: the others cancel in pairs when the controls are off.

L3-4 — multiply, swap, un-multiply

Shift-and-add, $ax = \sum_i x_i (a2^i)$, gives an *out-of-place* product: CMULT(a) is n modular additions of $a2^i$, each controlled by c and x_i . Phase estimation needs U_a *in place*:



$$C-U_a = \text{CMULT}(-a^{-1}) \cdot \text{C-SWAP} \cdot \text{CMULT}(a)$$

One rung is $2n$ modular additions and n controlled swaps. The second multiplication only erases the input: uncomputation, made possible by a^{-1} .

L3-4 in Qiskit

```
def c_mult_acc(qc, c, A, x, y, sign, flag, N):
    for i in range(len(x)):
        c_add_mod(qc, [c, x[i]], (A << i) % N, y, sign, flag, N)

def c_ua(qc, c, A, x, y, sign, flag, N):
    c_mult_acc(qc, c, A % N, x, y, sign, flag, N)
    for i in range(len(x)):
        qc.cswap(c, x[i], y[i])
    B = pow(A, -1, N)  # classical inverse, gcd(A,N)=1
    c_mult_acc(qc, c, (-B) % N, x, y, sign, flag, N)
```

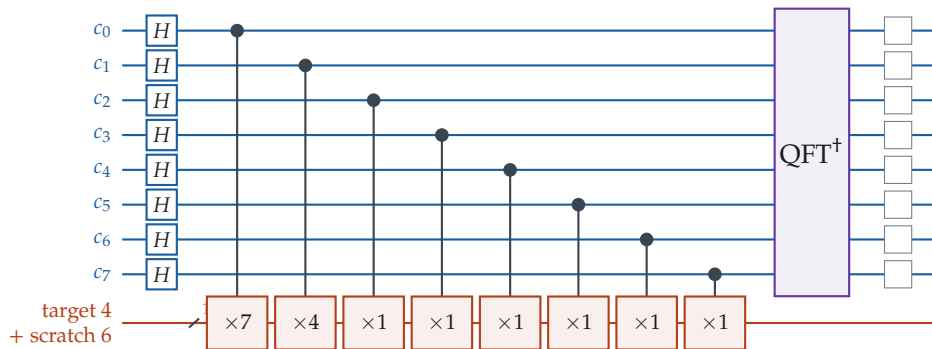
Why the inverse exists

$\gcd(a, N) = 1$, so $a^{-1} \bmod N$ exists; it is computed classically.

Checked

$|x\rangle|0\rangle \mapsto |ax \bmod N\rangle|0\rangle$ for every x , every a coprime to N , both control values.

L5 — the whole circuit, $N = 15, a = 7$



$4n + 2 = 18$ **qubits**

8 counting, 4 target, 4 scratch, sign and flag.

Rung i multiplies by $7^{2^i} \bmod 15$: 7, 4, 1, 1, ...

The circuit keeps the $\times 1$ rungs — skipping them would use r to build the circuit that finds r .

L5 in Qiskit

```
def order_circuit(A, N, t=None):
    n = math.ceil(math.log2(N))
    t = t if t is not None else 2 * n
    ctr, tgt = QuantumRegister(t, "ctr"), QuantumRegister(n, "tgt")
    anc = QuantumRegister(n, "anc")
    sf = QuantumRegister(2, "sf")          # sign qubit, flag qubit
    out = ClassicalRegister(t, "out")
    qc = QuantumCircuit(ctr, tgt, anc, sf, out)
    qc.h(ctr)                             # Hadamard layer
    qc.x(tgt[0])                           # target = |1>
    for i in range(t):                    # the ladder: t rungs
        c_ua(qc, ctr[i], pow(A, 2**i, N), tgt, anc, sf[0], sf[1], N)
    qc.append(qft(t).inverse(), ctr)      # inverse QFT
    qc.measure(ctr, out)
    return qc
```

Five short functions

qft, add_const,
c_add_mod, c_ua,
order_circuit — each a
product of the one
before — make the
complete order-finding
circuit.

Resolved. The box is modular multiplication, built from Fourier adders, with uncomputation at two levels. Next: run it.

Act V — simulation, and reading the answer

Act V — running it on Aer

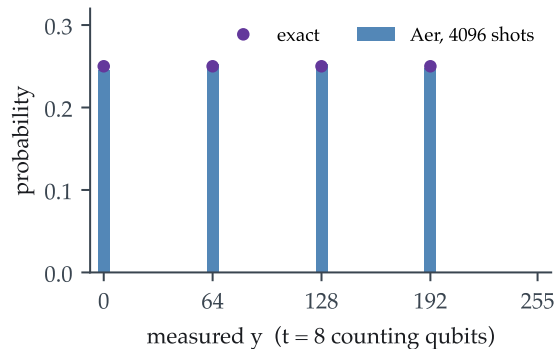
Difficulty 5. The measurement returns one integer y . How is r read from it, and how do we know the circuit is right?

```
from qiskit import transpile
from qiskit_aer import AerSimulator
from shor_essentials import order_circuit, order_from_counts

sim = AerSimulator(seed_simulator=2026)
qc = order_circuit(7, 15)                                # a = 7, N = 15: 18 qubits
counts = sim.run(transpile(qc, sim), shots=4096).result().get_counts()
r = order_from_counts(counts, 7, 15, t=8)
print(qc.num_qubits, r)
```

prints 18 4: 18 qubits, order $r = 4$. `transpile` rewrites the circuit into the simulator's gate set; 4096 shots take about 2 s.

$N = 15$: four sharp peaks



The exact distribution

The target takes r values, one per residue $x_0 = x \bmod r$:

$$P(y) = \frac{1}{4^t} \sum_{x_0 < r} \left| \sum_m e^{-2\pi i m r y / 2^t} \right|^2$$

Aer's exact probabilities equal it.

Samples vs exact: the total variation distance $\frac{1}{2} \sum_y |\hat{p}(y) - p(y)|$ is 0.006. A perfect sampler exceeds 0.032 only 0.01% of the time at 4096 shots — the *shot-noise bound*.

From y to r to the factors

Each peak is $y \approx 2^t s/r$. Continued fractions give s/r in lowest terms; its denominator is a candidate r , checked with $a^r \equiv 1$.

y	shots	$y/2^8$	candidate r
64	1037	$1/4$	4 ✓
128	1037	$1/2$	2 ($7^2 \not\equiv 1$)
192	1015	$3/4$	4 ✓
0	1007	0	—

Then Euclid

$r = 4$, and $7^2 = 49 \equiv 4 \not\equiv -1$:
 $\gcd(7^2 - 1, 15) = 3$, $\gcd(7^2 + 1, 15) = 5$.

One shot gives r with probability $1/2$ here (measured 0.501): about two runs expected. A failed shot costs a run, never a wrong answer.

Post-processing in code

```
def order_from_counts(counts, A, N, t):  
    for bits in sorted(counts, key=counts.get, reverse=True):  
        y = int(bits, 2)  
        if y == 0:  
            continue  
        r = Fraction(y, 2**t).limit_denominator(N - 1).denominator  
        if pow(A, r, N) == 1:  
            return r  
    return 0
```

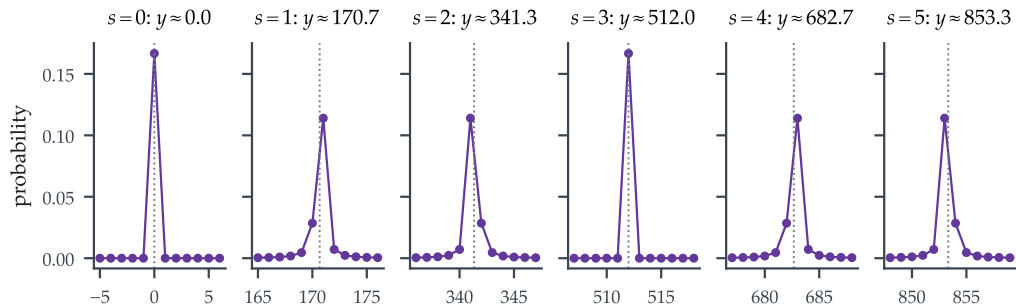
Continued fractions in one call

`limit_denominator(N-1)`
returns the best fraction
with denominator $< N$: the
continued-fraction
convergent.

Failure modes, all caught

$y = 0$ or $\gcd(s, r) > 1$: a
divisor of r — take another
shot. r odd or $a^{r/2} \equiv -1$:
pick another a (probability
 $\leq 1/2$).

When r does not divide 2^t : $N = 21, a = 2, r = 6$



Continued fractions at work

$y = 171$: $\frac{171}{1024} = [0; 5, 1, 84, 2]$,
 convergents $\frac{1}{5}, \frac{1}{6}, \dots$ $\frac{1}{6}$ gives $r = 6$, and
 $2^6 \equiv 1 \pmod{21}$.

Why $t = 2n$

Legendre: if $|y/2^t - s/r| < 1/(2r^2)$,
 then s/r is a convergent; $2^t \geq N^2$
 guarantees it. The peaks smear:
 $P = 0.322$ per shot.

Act VI — counting qubits, Toffolis and T gates

Act VI — the counts follow from the construction

Difficulty 6. How many qubits and gates does the circuit need, and which of them dominate the fault-tolerant cost?

level	qubits	Toffoli	controlled phases
controlled ADD_N (10 QFTs, $m = n + 1$)	$n + 4$	$2m + 3$	$5m(m - 1) + 2m$
one rung $C-U_a$	$2n + 3$	$4n^2 + 11n$	$2n(5m(m - 1) + 2m)$
U_{Shor} : $2n$ rungs + QFT ⁺	$4n + 2$	$8n^3 + 22n^2$	$\approx 20n^4$

Toffolis: $2n$ rungs $\times$ ($2n$ additions $\times$ ($2m + 3$) + n controlled swaps) = $8n^3 + 22n^2$. They come from the doubly-controlled phases (2 each) and one triply-controlled NOT per addition (3).

$O(n)$ qubits, $O(n^3)$ Toffolis — and $O(n^4)$ controlled phases. Every count equals its formula exactly for $n = 4, \dots, 8$ (backup).

$N = 15$: where the T gates go

Exact in Clifford+T

864 Toffolis	6 048 T
$\pi/4$ phases (CS, T)	8 261 T

Counted by `resources.count`, which walks the circuit and prices each gate by its standard decomposition.

To be synthesised

12 855 small rotations	$\approx 964\,000$ T
------------------------	----------------------

Each of the R rotations is synthesised to error $10^{-3}/R$: $\approx 3 \log_2(R/10^{-3})$ T each.

Total $\approx 978\,000$ T: 98.5% from rotations.

The Toffolis are not the problem. The rotations of the Fourier arithmetic are.

This circuit, from the verified formulas

8 194 qubits, 6.9×10^{10} Toffolis

exact QFTs: $\approx 2 \times 10^{17}$ T

approximate QFTs: $\approx 1 \times 10^{16}$ T

still dominated by rotations

Reported by the state of the art

Gidney–Ekerå 2021: $\approx 6\,200$ qubits,
 2.7×10^9 Toffolis — ripple-carry, win-
dowed.

Gidney 2025: $\approx 1\,400$ qubits, 6.5×10^9 Toffolis per run — approximate residue arithmetic.

***Resolved.** The textbook circuit is polynomial but enormous, and most of its cost is rotations. The optimisations of Act VII remove them first.*

Act VII — the optimisations, one idea each

Act VII — what each optimisation removes

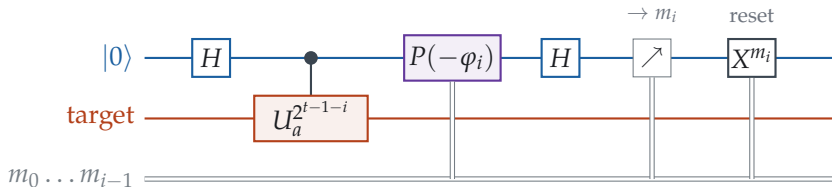
Difficulty 7. Which parts of the textbook circuit are expensive, and what replaces them?

optimisation	removes	measured effect
semiclassical QFT	the counting register	$4n + 2 \rightarrow 2n + 3$ qubits
ripple-carry adder	all rotations in the arithmetic	$T \div 19$ ($n = 4$) ... $\div 99$ ($n = 12$)
temporary AND	the T cost of uncomputing	$14 T \rightarrow 4 T$ per pair
windowing	most modular additions	$n \rightarrow \lceil n/w \rceil$ per multiply
approximate QFT	the smallest rotations	-34% rotations ($n = 8$), lossy

All implemented and verified in `shor_qiskit/`. The coset representation and measurement-based uncomputation are in the backup.

One counting qubit: the semiclassical QFT

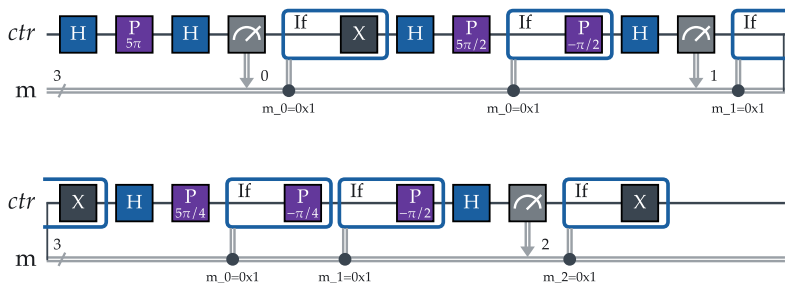
Every rotation of $\text{QFT}^\dagger$ is controlled by a qubit that is *about to be measured*. Measure it first and control classically: the distribution is unchanged (deferred measurement).



$$\varphi_i = \pi \sum_{j < i} m_j / 2^{i-j}, \quad \text{repeat for } i = 0, \dots, t-1 \text{ (heaviest rung first)}$$

One recycled qubit replaces all $2n$ counting qubits: $4n + 2 \rightarrow 2n + 3$. The controlled phases become classically controlled single-qubit phases.

In Qiskit: a dynamic circuit



The loop on its own, with phase gates as the rungs ($y = 5, t = 3$): Aer reads back 101 on every shot.

`if_test` is **mid-circuit measurement and feed-forward**: the classical bits already measured decide which phases fire. Hardware runs this as a dynamic circuit.

The loop in code

```
def semiclassical_iqft(qc, ctrl, out, rungs):
    t = len(rungs)
    assert len(out) >= t, "one classical bit per rung"
    for i in range(t):
        qc.h(ctrl)
        rungs[t - 1 - i](qc, ctrl)           # heaviest rung first
        for j in range(i):                  # the inverse QFT, classically
            with qc.if_test((out[j], 1)):
                qc.p(-math.pi / 2 ** (i - j), ctrl)
        qc.h(ctrl)
        qc.measure(ctrl, out[i])
        with qc.if_test((out[i], 1)):        # reset for the next rung
            qc.x(ctrl)
```

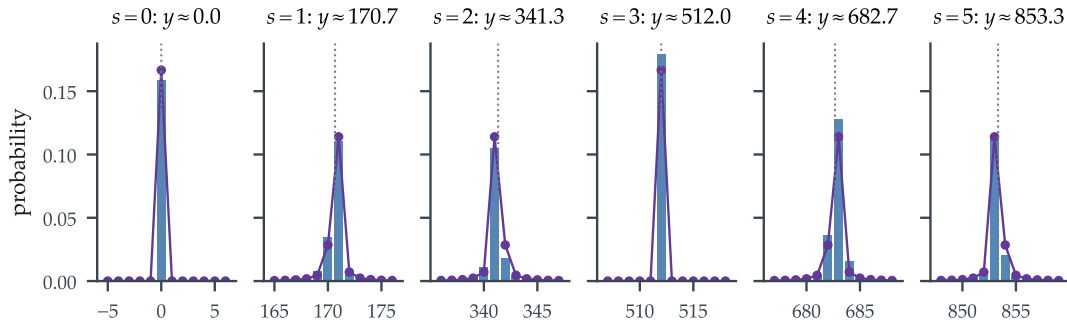
One loop, both problems

The rungs are callbacks: the same function drives factoring and ECDLP.

Checked exactly

Enumerating the measurement branches, its distribution equals the full circuit's to 10^{-13} .

$N = 21$ on 13 qubits instead of 22



bars: Aer, 1024 shots of the one-counting-qubit circuit; dots: the exact distribution around each peak $s \cdot 2^{10}/6$.

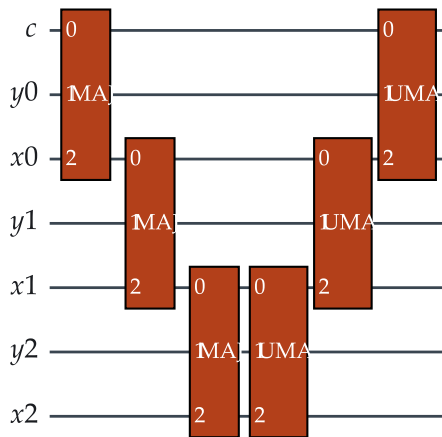
Same distribution

total variation distance 0.075, within the shot-noise bound 0.108.

Same success

one shot gives $r = 6$ with probability 0.322 exact, 0.313 measured; $21 = 7 \times 3$.

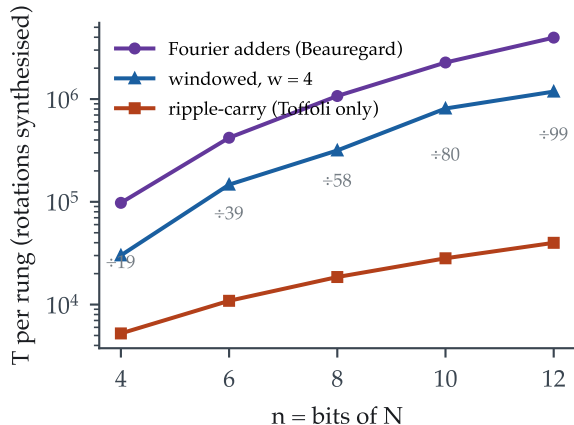
Toffoli arithmetic: the ripple-carry adder



Cuccaro et al.: **MAJ** blocks carry up, **UMA** blocks write the sum on the way down. Each is one Toffoli and two CNOTs.

$2m$ Toffolis, one carry qubit, **no rotations**. The seven blocks of the modular adder stay the same; only the adder inside changes.

Toffolis are cheaper than rotations



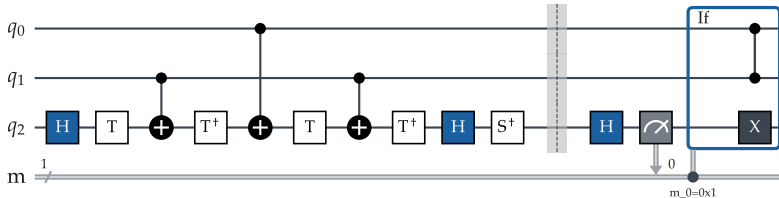
One rung, T gates with every rotation synthesised.

Ripple-carry: 7–8 $\times$ **more** Toffolis, but no rotations.

T: $\div 19$ at $n = 4$, $\div 99$ at $n = 12$.

The gap grows like n : rotations scale as n^4 , Toffolis as n^3 .

The temporary AND: uncomputation for free



Compute with 4 T; uncompute by an X-basis measurement and a classically controlled CZ.

Why it works

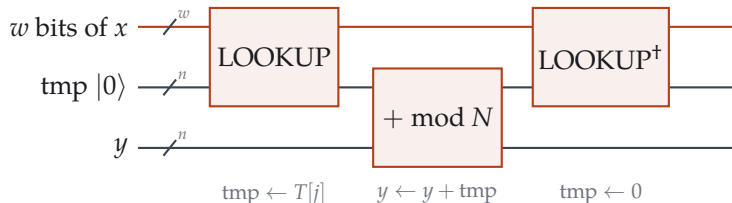
The target starts in $|0\rangle$: 4 T, not 7. An X-basis measurement reveals nothing about a, b ; a Clifford fixes the phase.

What it saves

A compute/uncompute pair drops from 14 T to 4: a table lookup costs $3.5\times$ fewer T.

Windowing: look the answer up, then add once

Take w bits j of x at a time, and **look up** their contribution $T[j] = a j 2^i \bmod N$ from a classical table:



The lookup is **unary iteration**: a tree of Toffolis walks the $L = 2^w$ addresses, $2(L - 1)$ Toffolis, and loads each entry with CNOTs.

$n \rightarrow \lceil n/w \rceil$ additions per multiplication.
One rung at $n = 12, w = 4$: T drops from 4.0×10^6 to 1.2×10^6 .

Approximate QFT: fewer rotations, but errors add up

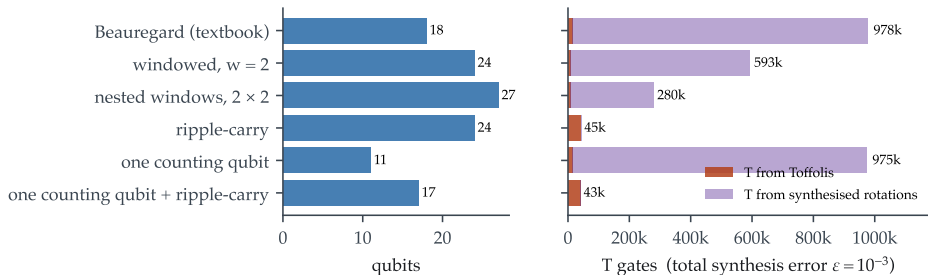
Drop every $\text{CP}(\pi/2^k)$ with $k > d$, in every adder's QFT. On $N = 15$:

QFTs	small rotations	distance from exact	P(one shot gives r)
exact	12 855	0.006	0.501
$d = 4$	12 837	0.006	0.501
$d = 3$	10 905	0.138	0.451
$d = 2$	7 050	0.594	0.245

The circuit holds 320 Fourier additions, each with its own approximation error; they accumulate. The shot-noise bound is 0.032.

*One error budget is shared by every approximation, so d must grow like $\log_2(\#\text{additions} / \varepsilon)$.
At $n = 8$, $d = 4$ already removes a third of the rotations.*

All together: $N = 15$



In raw Toffolis the ripple-carry versions look $7\times$ worse; priced with synthesis they win by more than $20\times$. One counting qubit + ripple-carry: 17 qubits and 42 600 T, against 18 qubits and 978 000 T.

Resolved. Remove rotations first, then qubits, then additions. (Modern estimates also combine windowing with ripple-carry adders; not built here.)

Act VIII — the elliptic-curve discrete logarithm

Act VIII — ECDSA keys are discrete logarithms

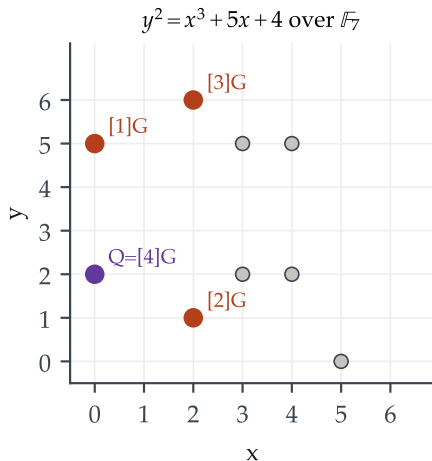
Difficulty 8. An ECDSA or ECDH key pair is a secret number d and a public point $Q = dG$ on a curve. Recovering d from Q is the elliptic-curve discrete logarithm. Does the same circuit solve it?

The points (x, y) with $y^2 = x^3 + ax + b \pmod{p}$, plus a point at infinity O , form a group. Adding two points:

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}, \quad x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1.$$

*Point addition needs a modular **division**. That is the whole cost difference from factoring.*

A toy curve



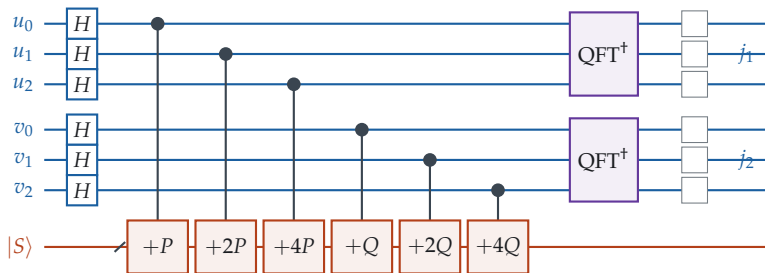
$y^2 = x^3 + 5x + 4$ over $\mathbb{F}_7$: nine points and O .

$kP = P + \dots + P$ (k times). $G = (0, 5)$ has order $r = 5$:

$$G, 2G, 3G, 4G, 5G = O.$$

The public key $Q = (0, 2) = 4G$ hides $k = 4$.

Two registers, one accumulator



$$\frac{1}{2^m} \sum_{u,v} |u\rangle |v\rangle |S + uP + vQ\rangle = \frac{1}{2^m} \sum_{u,v} |u\rangle |v\rangle |S + (u + kv)P\rangle$$

The accumulator depends on (u, v) only through $u + kv \bmod r$; two inverse QFTs turn that into (j_1, j_2) . S is a fixed start point: affine formulas cannot represent O , and hitting it is a rare exceptional case (probability $O(1/p)$); the toy oracle gives O its own code.

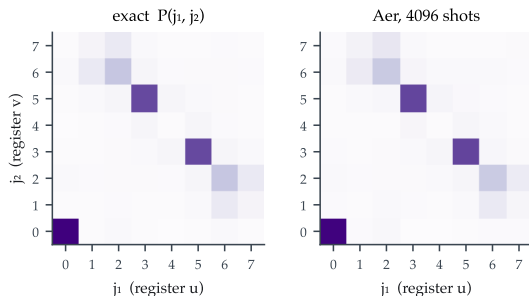
ECDLP in Qiskit

```
def ecdlp_circuit(curve, P, Q, m, S):
    n = curve.p.bit_length()
    u, v = QuantumRegister(m, "u"), QuantumRegister(m, "v")
    pt = QuantumRegister(2 * n, "point") # x, then y
    qc = QuantumCircuit(u, v, pt, ClassicalRegister(m, "j1"), ClassicalRegister(m, "j2"))
    for i in range(2 * n): # accumulator <- S
        if ((S.x + (S.y << n)) >> i) & 1:
            qc.x(pt[i])
    qc.h(u)
    qc.h(v)
    for i in range(m): # controlled +[2^i]P and +[2^i]Q
        permutation(qc, pt, point_perm(curve, curve.mul(2**i, P), n), ctrls=[u[i]])
        permutation(qc, pt, point_perm(curve, curve.mul(2**i, Q), n), ctrls=[v[i]])
    qc.append(qft(m).inverse(), u)
    qc.append(qft(m).inverse(), v)
    qc.measure(u, qc.cregs[0])
    qc.measure(v, qc.cregs[1])
    return qc
```

Table oracle: a permutation of the toy curve's points (12 qubits, on Aer; equals the library).

Arithmetic oracle: 69 qubits, checked on every basis state.

On Aer: the key comes out



Same distribution

distance from exact 0.031 (bound 0.054).
One shot yields k with probability 0.611
(measured 0.610).

Same one-qubit trick

With the semiclassical QFT both registers share one qubit: on a 13-point curve mod 11, $k = 9$ recovered on 9 qubits instead of 16.

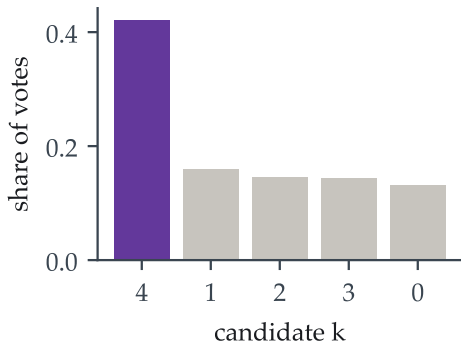
Post-processing: the key from (j_1, j_2)

A peak satisfies $j_2 r / 2^m \equiv k (j_1 r / 2^m) \pmod{r}$. Round, and divide:

$$a_i = \text{round}(j_i r / 2^m), \quad k = a_2 a_1^{-1} \pmod{r}.$$

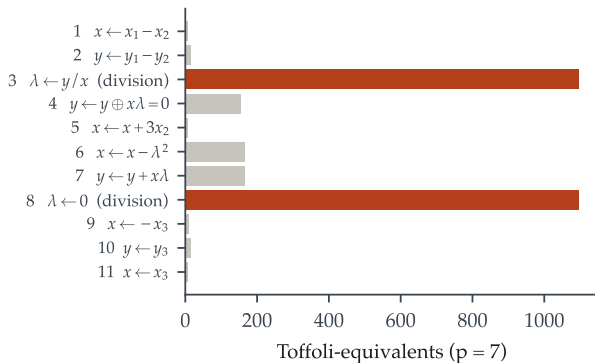
Every shot votes; the winner is checked with $kG = Q$.

Code: `ec_classical.ecdlp_postprocess(backup)`. It also tries ± 1 around the rounding when 2^m is not a multiple of r (Ekerå).



$k = 4$ takes 42% of the vote, against 13–16% for each wrong key: a signal, not luck.

One point addition is eleven reversible steps



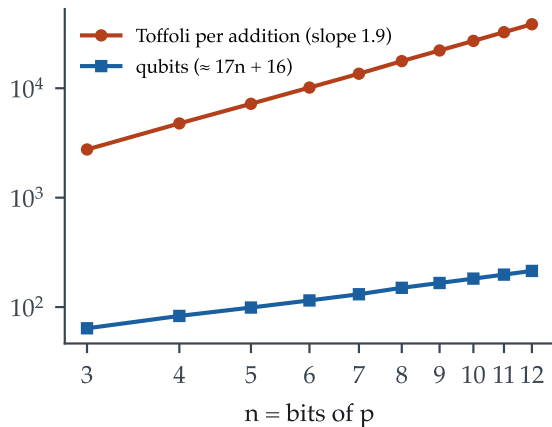
In place, no garbage

Step 3 divides to get λ . Step 4 clears y , since $x\lambda = y$. Step 8 is the *same* division; by then $y/x = \lambda$ again, so it **clears** λ — uncomputation.

A division is Kaliski's binary extended-Euclid inversion: $2n$ rounds of shifts and subtractions.

The two divisions are 80% of the 2 744 Toffolis of one addition ($p = 7$).

ECDLP costs: $O(n^2)$ per addition, $O(n^3)$ overall



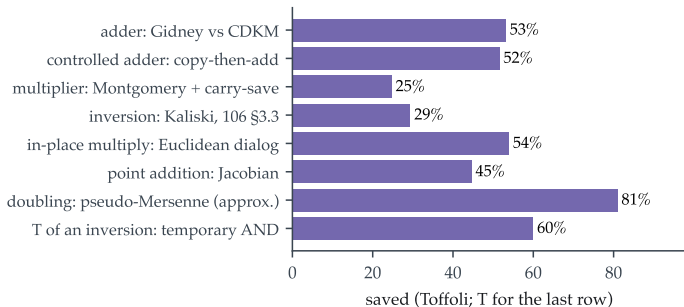
Why

A division: $2n$ rounds of $O(n)$ Toffolis.
One addition: $O(n^2)$ (measured slope 1.9). Shor adds one point per bit of u and v : $2n$ additions, $O(n^3)$ Toffolis on $O(n)$ qubits.

No rotations

The arithmetic is Toffolis and CNOTs only. The inverse QFTs become classically controlled phases.

ECDLP optimisations, measured



From eprint 2026/106 and 2026/1128, each on a fixed task, correctness re-checked.

Pseudo-Mersenne doubling is approximate (3% failures). Jacobian coordinates avoid the division but leave $3n$ garbage qubits per addition. The carry-save multiplier trades $4\times$ the qubits for depth.

Resolved. The same circuit solves the ECDLP. Its box, a point addition, is costlier than a multiplication because of the division — and again all Toffolis.

Act IX — why ECDSA falls before RSA at the same security

Act IX — what an attacker must solve

Difficulty 9. RSA-3072 and ECDSA on P-256 are rated equally secure against classical computers. Are they equally hard for Shor?

RSA

public key (N, e) with $N = pq$
factor $N \Rightarrow$ private exponent d
Shor: order finding, $n = \log_2 N$ bits

ECDSA / ECDH

public key $Q = dG$ on a curve over $\mathbb{F}_p$
discrete logarithm $\Rightarrow$ private key d
Shor: two registers, $n = \log_2 p$ bits

Shor recovers both private keys. The question is only how much each costs.

Same classical security, very different key sizes

Factoring: sub-exponential

The number field sieve runs in $\exp((1.92 + o(1))(\ln N)^{1/3}(\ln \ln N)^{2/3})$:
RSA needs long keys.

ECDLP: exponential

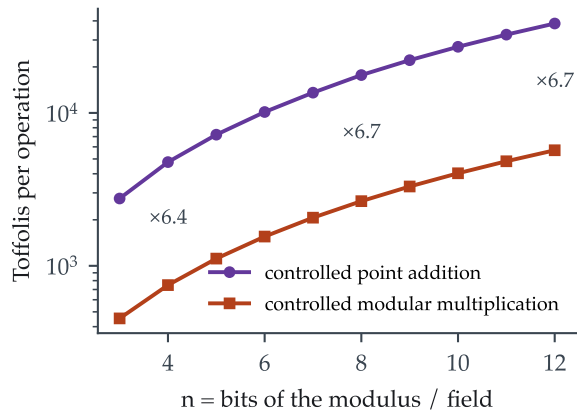
No sieve works on a good curve; the best attack is generic — Pollard's ρ , $\Theta(2^{n/2})$.

security	RSA	curve
112	2048	224
128	3072	256
192	7680	384
256	15360	521

key sizes in bits (NIST SP 800-57)

At matched classical security the curve key is $12\times$ shorter. That is why elliptic curves are used.

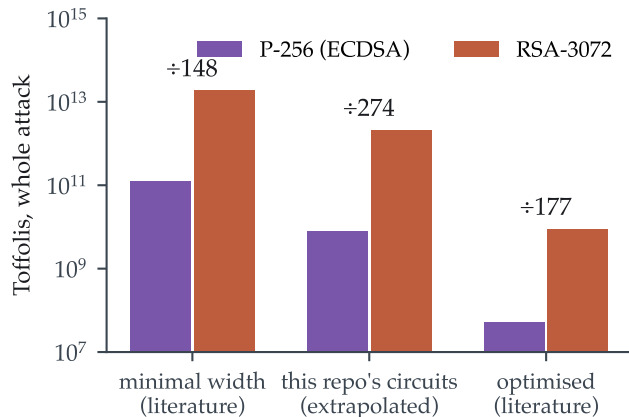
The price of one group operation



Both attacks do $2n$ controlled group operations: $\Theta(n)$ qubits, $\Theta(n^3)$ Toffolis. Only the constant differs.

Measured on this repo's circuits, at equal n : one point addition costs **6.1–6.7** controlled multiplications — constant in n . The division is the price.

At 128-bit security



Toffolis

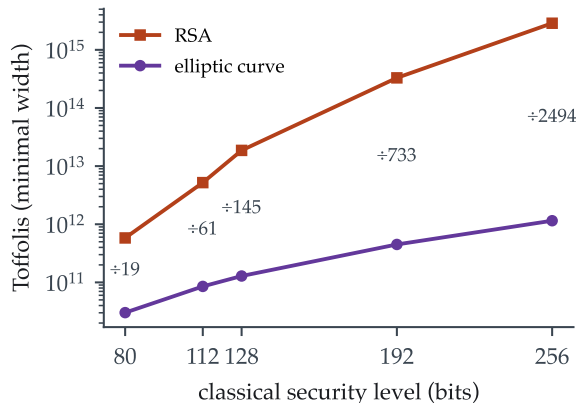
$(3072/256)^3 = 1\,728$, and
 $1\,728/6.7 \approx 260$:

this repo's circuits $\approx 270\times$ fewer
for P-256; minimal-width
literature $148\times$; optimised
literature $\approx 180\times$.

Qubits

depend on the construction: $2.6\times$
fewer in like-for-like circuits, but
RSA-2048 now fits in $\approx 1\,400$
(Gidney 2025).

The gap grows with security



At σ bits of classical security:

RSA needs $n = \Theta(\sigma^3 / \log^2 \sigma)$
 $\Rightarrow$ Shor costs $\Theta(\sigma^9 / \log^6 \sigma)$.

A curve needs $n = 2\sigma$
 $\Rightarrow$ Shor costs $\Theta(\sigma^3)$.

Minimal-width formulas: $19\times$ at 80 bits,
 $145\times$ at 128, $733\times$ at 192. Our circuits:
 $\approx 1\,300\times$ at 192.

What it means

Elliptic curves are the cheaper target

The short key that makes ECC efficient classically is exactly what makes it the easier quantum target: about two orders of magnitude fewer Toffolis than RSA at the same classical security. Optimised estimates keep the gap: $\approx 5 \times 10^7$ Toffolis for a 256-bit curve (Litinski 2023) against $\approx 9 \times 10^9$ for RSA-3072 (Gidney–Ekerå’s formula).

Key exchange (ECDH)

Traffic recorded today can be decrypted later: *harvest now, decrypt later*.

Signatures (ECDSA)

Once a large machine exists, anyone can forge. Migrate to ML-KEM and ML-DSA (FIPS 203, 204).

Resolved. At equal classical security, ECDSA falls first: same cubic cost, a $12\times$ shorter key, and only a $\approx 7\times$ heavier group operation.

Five things to keep

1. **Shor is phase estimation of a group operation.** The quantum part is short; the cost is reversible arithmetic.
2. **Uncomputation is mandatory.** Garbage enters $P(y)$ through $\langle g(x') | g(x) \rangle$; if it records x , the peaks vanish.
3. **The price is T gates.** A Toffoli costs 7; a small rotation $\approx 3 \log_2(1/\epsilon)$ after synthesis (Solovay–Kitaev, Ross–Selinger).
4. **Optimise in order.** Toffoli arithmetic removes rotations, the semiclassical QFT removes the counting register, windowing removes additions.
5. **Elliptic curves fall first.** Both attacks cost $\Theta(n^3)$ Toffolis; P-256's n is $12\times$ smaller than RSA-3072's: about $150\text{--}270\times$ cheaper.

Backup — why the QFT ends with swaps

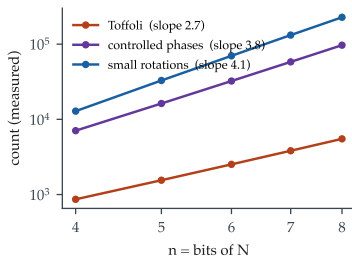
Output qubit q needs the phase $e^{2\pi i j 2^q / 2^n}$, which depends only on the lowest $n - q$ bits of j . The in-place loop builds that factor on qubit $n - 1 - q$.

phase on each qubit ($\times 2\pi$), input $ 1\rangle$, $n = 3$	qubit 0	qubit 1	qubit 2
required: $j 2^q / 2^n$	0.125	0.250	0.500
the loop without the swaps	0.500	0.250	0.125
qft(), with the swaps	0.125	0.250	0.500

The swaps are bookkeeping (3 CNOT, 0 T): before a measurement, inside an adder, or in the semi-classical QFT the reversal is absorbed by relabelling.

Backup — measured counts against the formulas

n	N	qubits	$4n + 2$	Toffoli	$8n^3 + 22n^2$	CP	formula
4	15	18	18	864	864	7 068	7 068
5	21	22	22	1 550	1 550	16 245	16 245
6	33	26	26	2 520	2 520	32 322	32 322
7	65	30	30	3 822	3 822	58 107	58 107
8	129	34	34	5 504	5 504	96 888	96 888



Backup — counting in code

```
import resources as R
from shor_essentials import order_circuit

r = R.count(order_circuit(7, 15))
print(r.qubits, r.toffoli, r.cphase, r.rotations)
print(r.T, r.T_with_rotations(1e-3))
```

prints 18 864 7068 12855 and 14309 978434.

gate	priced as
CCX, CSWAP	1 Toffoli, 7 T
MCX, k controls	$2k - 3$ Toffolis
MCP, k controls	$2(k - 1)$ Toffolis + CP
CP(θ)	2 CNOT + $3 P(\pm\theta/2)$
$P(\theta), \theta \notin \frac{\pi}{4}\mathbb{Z}$	1 small rotation
temporary AND	4 T, uncompute 0

Backup — Draper's adder in Qiskit

```
def add_const(qc, X, y):
    n = len(y)
    qc.append(qft(n), y)
    for i in range(n):
        qc.p(2 * np.pi * X * 2.0 ** (i - n), y[i])
    qc.append(qft(n).inverse(), y)

def c_add_const(qc, ctrls, X, y):
    n = len(y)
    qc.append(qft(n), y)
    for i in range(n):
        qc.mcp(2 * np.pi * X * 2.0 ** (i - n), ctrls, y[i])
    qc.append(qft(n).inverse(), y)
```

Backup — the modular adder in Qiskit

```
def c_add_mod(qc, ctrls, X, y, sign, flag, N):
    yw = list(y) + [sign]          # (n+1)-wide value register
    c_add_const(qc, ctrls, X, yw)    # (1) + X          (controlled)
    add_const(qc, -N, yw)           # (2) - N
    qc.cx(sign, flag)              # (3) sign -> flag
    c_add_const(qc, [flag], N, yw)  # (4) + N if flag
    add_const(qc, -X, yw)           # (5) - X
    qc.mcx(list(ctrls) + [sign], flag) # (6) clear flag from the answer
    qc.x(flag)
    add_const(qc, X, yw)            # (7) + X
```

Backup — the windowed multiplier and the ripple-carry adder in Qiskit

```
def windowed_c_mult_acc(qc, c, k, x, y, sign, flag, N, t, anc, w):
    n = len(y)
    for i in range(0, len(x), w):
        win = list(x[i:i + w])
        # window value j contributes  $(k * j * 2^i) \bmod N$ 
        table = [((k * j) << i) % N for j in range(2**len(win))]
        lookup_ui(qc, c, win, t[:n], table, anc[:len(win)])
        add_quantum_mod(qc, t[:n], y, sign, flag, N)
        lookup_ui(qc, c, win, t[:n], table, anc[:len(win)]) # uncompute
```

```
def rc_add(qc, x, y, carry, ctrls=()):
    n = len(y)
    assert len(x) == n, "x and y must be the same width"
    _maj(qc, carry, y[0], x[0], ctrls)
    for i in range(n - 1):
        _maj(qc, x[i], y[i + 1], x[i + 1], ctrls)
    for i in range(n - 1, 0, -1):
        _uma(qc, x[i - 1], y[i], x[i], ctrls)
    _uma(qc, carry, y[0], x[0], ctrls)
```

Backup — post-processing and the garbage demo in code

Factor, retrying (run: any circuit → counts)

```
while True:
    a = random.randrange(2, N)
    if (d := math.gcd(a, N)) > 1:
        return d                # lucky gcd
    t = 2 * math.ceil(math.log2(N))
    r = order_from_counts(run(order_circuit(a, N)), a, N, t)
    if r and r % 2 == 0:
        d = math.gcd(pow(a, r // 2, N) - 1, N)
        if 1 < d < N:
            return d
```

The garbage demo: one extra line

```
for i in range(t):
    c_ua(qc, ctr[i], pow(A, 2**i, N), list(tgt), list(anc), sf[0], sf[1], N)
if garbage == "x":
    for i in range(t):
        qc.cx(ctr[i], g[i])          # a copy of x left behind
qc.append(qft(t).inverse(), ctr)
```

ECDLP: votes for k

```
votes, q = {}, 1 << bits
for (j1, j2), c in items:
    for da in range(-search, search + 1):
        for db in range(-search, search + 1):
            a1 = (round(j1 * order / q) + da) % order
            a2 = (round(j2 * order / q) + db) % order
            if gcd(a1, order) != 1:
                continue
            k = a2 * pow(a1, -1, order) % order
            w = c / (1 + abs(da) + abs(db)) ** 2
            votes[k] = votes.get(k, 0.0) + w
return sorted(votes.items(), key=lambda kv: -kv[1])
```

ECDLP end to end

```
# the toy curve  $y^2 = x^3 + 5x + 4 \pmod{7}$ , with G of order 5
from ec_classical import CLASSIQ as curve, CLASSIQ_G as G
from ec_shor import solve

Q = curve.mul(4, G)                # public key: Q = [4]G
# build the circuit, run it on Aer, post-process the counts
k, counts, info = solve(curve, G, Q, order=5)
print(k, info["qubits"])
```

Backup — how the circuits are verified

Arithmetic, exactly

Every block on every basis input; every ancilla checked back to $|0\rangle$ where it is freed.

Distributions, exactly

Closed-form $P(y)$ against Aer's statevector probabilities: equal to 10^{-14} .

Samples, statistically

Every histogram within the distance a perfect sampler reaches at that shot count.

Test the distribution, not the factors: a broken circuit gives a flat histogram, and the classical checks can still print the right factors by luck.

Backup — two more optimisations

Coset representation (Zalka; Gidney-Ekerå)

Store k as $\sum_j |jN + k\rangle$ on $n + c$ qubits: a **plain** adder then does modular addition, error $\approx 2^{-c}$ per addition. Removes the comparison and the flag; pays in register width — it wins only at large n .

Measurement-based uncomputation

Uncompute a lookup by measuring its output in the X basis, then fix the phase left on the address: $2(L - 1)$ Toffolis become $O(\sqrt{L})$. For $L = 256$: $510 \rightarrow 45$.

Backup — elliptic curves against RSA, across key sizes

ECC bits	qubits	Toffoli	RSA bits	qubits	Toffoli
160	1 466	3.0×10^{10}	1 024	2 050	5.8×10^{11}
224	2 042	8.4×10^{10}	2 048	4 098	5.2×10^{12}
256	2 330	1.3×10^{11}	3 072	6 146	1.9×10^{13}
384	3 484	4.5×10^{11}	7 680	15 362	3.3×10^{14}
521	4 719	1.1×10^{12}	15 360	30 722	2.9×10^{15}

Rows pair comparable classical security. Minimal-width logical counts:

Roetteler–Naehrig–Svore–Lauter (2017; $9n + 2\lceil \log_2 n \rceil + 10$ qubits, $(448 \log_2 n + 4090) n^3$ Toffolis) and Häner–Roetteler–Svore (2017; $2n + 2$ qubits). Error correction multiplies both sides by comparable factors.

Backup — reproduce every number

```
python3 -m venv venv && venv/bin/pip install -r requirements.txt
./run_tests.sh fast      # incl. semiclassical QFT and resource counts
./run_tests.sh full      # + end-to-end factoring on Aer
./run_tests.sh ec        # the ECDLP suite
venv/bin/jupyter lab notebooks/shor_walkthrough.ipynb
venv/bin/python presentation/make_figures.py  # this deck's figures
venv/bin/python presentation/make_listings.py # its code, checked
```

Backup — references

- P. W. Shor, SIAM J. Comput. 26 (1997). S. Beauregard, Quantum Inf. Comput. 3 (2003).
- T. G. Draper, quant-ph/0008033. S. Cuccaro, T. Draper, S. Kutin, D. Moulton, quant-ph/0410184.
- R. Griffiths, C.-S. Niu, Phys. Rev. Lett. 76 (1996). C. H. Bennett, IBM J. Res. Dev. 17 (1973). T. Toffoli, ICALP 1980.
- C. Dawson, M. Nielsen, quant-ph/0505030. N. J. Ross, P. Selinger, arXiv:1403.2975.
- C. Gidney, arXiv:1709.06648 (temporary AND); arXiv:1905.07682 (windowing); arXiv:2505.15917 (2025).
- C. Gidney, M. Ekerå, Quantum 5, 433 (2021). D. Litinski, arXiv:2306.08585 (2023).
- M. Roetteler, M. Naehrig, K. Svore, K. Lauter, ASIACRYPT 2017. T. Häner, M. Roetteler, K. Svore, arXiv:1611.07995.
- Kim et al., eprint 2026/106. A. Schrottenloher, eprint 2026/1128. NIST SP 800-57; FIPS 203, 204.